

Documentação final

Quero Trabalhar Backend

Versão refatorada para handoff técnico

Jefferson Bezerra
Maio de 2026

Esta versão foi refatorada para reduzir redundâncias, preservar decisões críticas e tornar a leitura mais útil para continuidade do backend e integração do frontend. Os contratos exatos continuam sendo responsabilidade do Swagger/OpenAPI; este documento explica o contexto, as decisões e os cuidados de manutenção.

Como usar este documento

A documentação foi reorganizada para manter as 22 seções principais, mas com texto mais enxuto. As seções 1 a 8 explicam origem, problemas, objetivos, arquitetura, segurança e domínio. As seções 9 a 16 aprofundam módulos e decisões técnicas. As seções 17, 18 e 22 orientam o frontend. As seções 19 a 21 registram trade-offs, roadmap e cuidados para continuidade.

Recurso	Papel
Swagger/OpenAPI	Fonte formal de endpoints, parâmetros, DTOs, filtros e respostas.
README	Guia de execução, ambiente, Docker, Swagger, testes e visão geral.
docs/testing.md	Explica a suíte de testes e como executá-la.
Postman	Validação manual de fluxos e autenticação.
Este documento	Explica decisões, regras, trade-offs e cuidados para backend/frontend.

1. Introdução

A API Quero Trabalhar Backend evoluiu de uma base funcional, mas exploratória, para um MVP mais organizado, seguro e documentado. O objetivo desta documentação é registrar o raciocínio por trás dessa evolução, para que a próxima pessoa que assumir o backend ou integrar o frontend entenda não apenas o que foi implementado, mas por que foi implementado dessa forma.

O projeto inicial já representava a ideia central do produto: conectar candidatos, recrutadores e oportunidades de emprego. Porém, ainda havia exposição direta de entidades JPA, localidade em texto livre, empresa como campo textual, risco de IDOR, tratamento de erros inconsistente, listagens sem paginação, poucos testes e documentação defasada.

Este documento não substitui Swagger nem README. Ele explica decisões de arquitetura, domínio, segurança, Localidade, trade-offs de MVP e cuidados de continuidade.

A decisão mais sensível da API é o tratamento de Localidade. Recursos públicos dependem de Localidade validada; LocalidadePendente é uma fila técnica temporária que permite salvar recursos internamente sem expô-los ao público enquanto a localização não for resolvida com segurança.

2. Para quem esta documentação foi escrita

2.1 Desenvolvedor backend

Para o backend, este documento funciona como guia de manutenção. Ele explica onde ficam as regras de negócio, por que os fluxos pessoais usam /me, como LocalidadePendente deve ser tratada, por que Empresa ainda não possui ownership formal e quais regras não devem ser quebradas em evoluções futuras.

2.2 Pessoa responsável pelo frontend

Para o frontend, esta documentação explica decisões que impactam a integração. Os contratos exatos de endpoints, parâmetros, DTOs, filtros e respostas devem ser consultados no Swagger/OpenAPI. Aqui são explicadas as regras de contexto: público, /me e admin; JWT; paginação; erros; DTO público/interno; e LocalidadePendente.

O ponto mais importante para o frontend é entender que empresas e oportunidades podem existir internamente e ainda não aparecer publicamente quando a localidade estiver pendente. Isso não deve ser tratado automaticamente como erro de cadastro.

2.3 Avaliadores e colaboradores

Para avaliadores e colaboradores, o documento registra a evolução do MVP e as decisões de escopo. Algumas funcionalidades importantes ficaram para pós-MVP por escolha consciente, como confirmação manual de localidade, notificação de pendência e ownership formal de Empresa.

3. Estado inicial da API

O estado inicial era uma API CRUD funcional, mas ainda exploratória. Existiam usuários, recrutadores, oportunidades, tipos de emprego, experiências, interesses e indicações. A stack já incluía Spring Boot, Spring Data JPA, Spring Security, JWT, BCrypt, H2 e validação, mas o desenho ainda misturava persistência, contrato HTTP e regra de negócio.

3.1 Características principais

- Controllers frequentemente recebiam ou retornavam entidades JPA.
- Services tinham pouca regra de negócio e muitas vezes apenas repassavam chamadas ao repository.
- Localidade era texto livre: cidade, estado e país não tinham modelo relacional.
- Empresa era apenas texto em recrutador ou oportunidade.
- Recrutador não estava bem integrado à identidade autenticada.
- Oportunidade não tinha dono claro nem vínculo seguro com recrutador autenticado.
- Listagens retornavam List, sem Page, Pageable ou Specification.
- Não havia tratamento global de erros nem documentação OpenAPI real.
- Docker, documentação de testes e suíte ampla de testes ainda não existiam.

3.2 Pontos aproveitáveis

Apesar dos problemas, a base inicial tinha valor. Ela já continha o domínio básico do produto, autenticação JWT, BCrypt, roles, H2 para desenvolvimento e alguns DTOs de usuário. A refatoração aproveitou essa base e reorganizou os pontos frágeis, em vez de reescrever tudo do zero.

4. Problemas encontrados

O diagnóstico inicial mostrou problemas de arquitetura, segurança, domínio, integração, documentação e qualidade. Esses problemas explicam as decisões tomadas no estado final.

4.1 Problemas arquiteturais

- Entidades JPA expostas como contrato HTTP.
- Poucos DTOs e pouca separação entre request, response e persistência.
- Services sem centralização forte de regra de negócio.
- Ausência de handler global para erros.
- Ausência de Specification para filtros dinâmicos.

4.2 Problemas de segurança

- Risco de IDOR por uso de usuariold ou IDs sensíveis em ações pessoais.
- Autenticação existente, mas autorização de domínio insuficiente.
- Possibilidade de payload controlar mais do que deveria, como papéis de usuário em alguns fluxos.
- Falta de padronização de 401 e 403.

4.3 Problemas de domínio

- Empresa como texto, sem identidade, vínculo ou localidade própria.
- Localidade como texto livre, sem validação ou normalização.
- Oportunidade sem dono claro e sem publicação controlada em nome de empresa.
- Candidato, experiências, interesses e indicações sem ownership consistente.
- TipoDeEmprego sem curadoria claramente consolidada.

4.4 Problemas de integração e operação

- Listagens sem paginação e filtros no backend.
- Erros heterogêneos e pouco previsíveis para frontend.
- README desatualizado, Swagger inexistente na prática e ausência de documentação técnica.
- Ausência de Docker e testes insuficientes.
- Problemas de acentuação e encoding em textos humanos.

5. Objetivos da refatoração

A refatoração teve como objetivo transformar uma base inicial funcional em uma API de MVP mais segura, compreensível, testável e integrável. O foco foi corrigir riscos reais sem apagar o que já funcionava.

5.1 Objetivos técnicos

- Separar contrato HTTP de modelo JPA com DTOs e Java Records.
- Manter controllers finos e concentrar regras de negócio nos services.
- Padronizar erros com `@RestControllerAdvice`, `StandardError`, `ValidationError` e `FieldMessage`.
- Introduzir paginação, filtros, `Pageable`, `Page`, `FilterDTO` e `Specification` nas listagens principais.
- Ampliar testes, documentar contratos e facilitar execução com Docker.

5.2 Objetivos de negócio e segurança

- Reduzir IDOR com endpoints `/me` e identidade derivada do JWT.
- Transformar Empresa em entidade real, com localidade e vínculo com recrutadores.
- Fazer Oportunidade nascer no contexto do recrutador autenticado.
- Criar catálogo moderado de `TipoDeEmprego`.
- Tratar Localidade como regra de visibilidade pública e `LocalidadePendente` como fila técnica temporária.

5.3 Escopo consciente de MVP

Algumas frentes foram identificadas, mas deixadas para pós-MVP: confirmação manual de localidade, notificação de recurso oculto por pendência, ownership formal de Empresa, painel operacional de `LocalidadePendente`, contrato próprio de paginação, revogação de token e preferências completas do candidato.

6. Evolução arquitetural

A arquitetura evoluiu de um CRUD direto para uma API em camadas. A mudança principal foi separar responsabilidades: controller como borda HTTP, DTO como contrato, service como dono da regra de negócio, repository como persistência, Specification como composição de filtros e handler global como ponto único de erro.

6.1 Responsabilidades finais

Camada	Responsabilidade
Controller	Receber requisições, validar borda, documentar endpoint e delegar ao service.
DTO	Definir contrato HTTP de entrada, saída e filtros.
Service	Aplicar regra de negócio, ownership, validações, transações e orquestração.
Repository	Executar persistência e consultas específicas.
Specification	Montar filtros dinâmicos e regras de visibilidade em consultas paginadas.
Security	Autenticar, fornecer identidade e proteger rotas sensíveis.
Exception Handler	Converter exceptions em respostas HTTP padronizadas.

6.2 Decisões que devem ser preservadas

- Entidade JPA não deve ser contrato da API.
- Controller não deve concentrar regra de negócio.
- Service deve proteger domínio, ownership e transações.
- Query param filtra dentro do escopo; não define escopo sensível.
- Specification pública deve excluir recursos sem Localidade validada antes da paginação.
- DTO público e DTO interno podem ser diferentes.

6.3 Módulo de Localidade como arquitetura própria

Localidade foi separada em componentes para evitar acoplamento: catálogo oficial, resolução por base interna, fallback Google Maps, conversão de resultado externo, registro de pendência, reprocessamento e scheduler. Essa separação impede que Google Maps ou LocalidadePendente contaminem controllers e contratos públicos.

7. Segurança e autorização

A segurança evoluiu de autenticação com JWT para autorização orientada ao domínio. A API não precisa apenas saber quem está logado; precisa validar se esse usuário pode executar a ação sobre o recurso específico.

7.1 Regra central

Autenticação identifica o usuário. Services e regras de domínio determinam o que ele pode fazer.

7.2 Contextos de acesso

Contexto	Uso	Regra principal
Público	Login, cadastro, catálogos, empresas e oportunidades públicas.	Não exige token, mas só expõe recursos publicáveis.
/me	Dados e recursos do usuário autenticado.	Identidade vem do JWT; cliente não envia usuarioid.
/api/admin/**	Gestão global e moderação.	Exige perfil ADMIN e não substitui fluxos pessoais.

7.3 JWT, 401 e 403

O login é feito em POST /login e retorna o JWT no header Authorization. Requisições protegidas devem enviar Authorization: Bearer <token>. 401 indica falha de autenticação; 403 indica usuário autenticado sem permissão.

7.4 Regras de domínio protegidas nos services

- Candidato só manipula próprias experiências, interesses e indicações permitidas.
- Recrutador só gerencia próprias oportunidades.
- Publicar como empresa exige vínculo aprovado.
- Admin modera tipos de emprego e vínculos empresa-recrutador.
- Fluxos pessoais devem preferir /me para reduzir risco de IDOR.

8. Evolução do domínio

O domínio evoluiu de entidades simples e campos textuais para um modelo mais próximo das regras reais do produto. A mudança central foi separar identidade, papéis, recursos, vínculos e visibilidade pública.

8.1 Modelo final em alto nível

- **Usuario:** base autenticável do sistema.
- **PerfilCandidato:** papel de quem busca oportunidades, gerencia experiências, interesses e indicações.
- **PerfilRecrutador:** papel de quem publica oportunidades e solicita vínculo com empresa.
- **Empresa:** entidade própria, com dados públicos, localidade e vínculo com recrutadores.
- **OportunidadeDeEmprego:** recurso publicado pelo recrutador autenticado, opcionalmente em nome de empresa.
- **TipoDeEmprego:** catálogo moderado, com aprovados e sugestões pendentes.
- **Localidade:** fonte oficial de visibilidade pública.
- **LocalidadePendente:** fila técnica temporária para recursos com localização ainda não resolvida.

8.2 Regras transversais

- Usuario autentica; perfis definem capacidades de negócio.
- Recursos pessoais usam o usuário autenticado como escopo.
- Empresa e Oportunidade públicas exigem Localidade validada.
- LocalidadePendente não é erro fatal nem recurso público.
- Admin modera o que precisa de controle global: usuários, tipos, experiências e vínculos.

9. Localidade e LocalidadePendente

Localidade é o módulo mais sensível da API porque controla visibilidade pública. No estado inicial, cidade, estado e país eram texto livre. No estado final, Localidade validada se tornou a fonte oficial; LocalidadePendente passou a registrar casos que ainda não puderam ser resolvidos com segurança.

Recurso público só aparece quando possui Localidade validada. LocalidadePendente permite salvar internamente, mas impede exposição pública.

9.1 Por que LocalidadePendente existe

Bloquear todo cadastro quando uma localidade não é resolvida criaria fricção. Publicar dados incertos prejudicaria o catálogo. A solução intermediária foi salvar o recurso internamente, registrar a pendência, ocultar do público e reprocessar depois.

9.2 Fluxo conceitual

1. Receber localidade por IDs estruturados ou texto livre.
2. Priorizar base interna oficial quando possível.
3. Usar Google Maps apenas como fallback encapsulado.
4. Criar ou reutilizar Localidade validada quando o resultado for seguro.
5. Criar LocalidadePendente quando houver falha, ausência ou ambiguidade.
6. Ocultar recurso em endpoints públicos enquanto pendente.
7. Permitir visibilidade interna ao dono quando fizer sentido.
8. Reprocessar pendências em segundo plano e promover para Localidade validada quando possível.

9.3 Responsabilidades internas

Componente	Papel
LocalidadeService	Catálogo oficial de países, estados e cidades. Não chama Google Maps.
LocalidadeResolucaoService	Orquestra a resolução no fluxo de cadastro/atualização.
ResolvedorLocalidadeBaseInterna	Tenta resolver usando dados oficiais internos.
ResolvedorLocalidadeGoogleMaps	Encapsula o fallback externo.
RegistroLocalidadePendenteService	Registra pendência quando não há resolução segura.
LocalidadePendenteReprocessamentoService	Tenta resolver pendências existentes sem duplicá-las.
Scheduler	Dispara reprocessamento conforme configuração.

9.4 Regras que não devem ser quebradas

- Google Maps é infraestrutura interna, não contrato público.
- Ambiguidade não deve ser escolhida automaticamente.
- LocalidadePendente não deve voltar como FK direta em Empresa ou Oportunidade.
- Endpoints públicos não devem expor pendência nem filtrar por statusLocalidade.
- DTO público não expõe pendência; DTO interno pode expor status quando útil ao dono.
- Reprocessamento trabalha sobre a pendência existente e não deve criar duplicatas.

9.5 Limites do MVP

Ficaram para pós-MVP: confirmação manual de sugestão de localidade, notificação ao usuário sobre recurso oculto, painel operacional de pendências, histórico terminal e revisão da exposição técnica nos DTOs internos.

10. Empresa e recrutador

Empresa deixou de ser texto e passou a ser entidade de domínio. Recrutador deixou de ser entidade desconectada da autenticação e passou a ser PerfilRecrutador associado ao Usuario. Essa mudança permitiu controlar vínculos, visibilidade pública, oportunidades por empresa e publicação em nome de empresa.

10.1 Modelo final

- Empresa possui identidade, dados públicos, localidade e status ativo.
- PerfilRecrutador representa o usuário que pode publicar oportunidades.
- Recrutador pode solicitar vínculo com empresa.
- Admin aprova ou recusa vínculo.
- Apenas vínculos aprovados devem aparecer publicamente e permitir publicação como empresa.

10.2 Regras principais

- Empresa pública exige Localidade validada, conforme regra da seção 9.
- Empresa interna pode existir mesmo com localidade pendente.
- Recrutador autenticado vem do JWT, não do request.
- PublicarComoEmpresa só é válido com vínculo aprovado.
- Recrutador vinculado não é automaticamente dono da empresa.

10.3 Ownership ficou para pós-MVP

O MVP possui vínculo moderado, mas não ownership formal de Empresa. Ainda não há dono da empresa, administradores empresariais, conta empresarial, login por CNPJ ou gestão interna de permissões por empresa. Qualquer evolução nessa direção deve revisar segurança, endpoints de empresa, vínculos e publicação de oportunidades.

10.4 Cuidados de manutenção

- Não voltar a tratar empresa como String.
- Não permitir representação pública sem vínculo aprovado.
- Não expor empresa pendente em endpoints públicos.
- Não confundir vínculo aprovado com ownership.
- Não adicionar login por CNPJ como simples campo sem revisar identidade e autorização.

11. Oportunidade de emprego

Oportunidade é o recurso central que conecta recrutadores, empresas e candidatos. No estado final, toda oportunidade nasce no contexto do recrutador autenticado, pode ser publicada em nome próprio ou de empresa vinculada, usa TipoDeEmprego aprovado, possui modalidade e depende da regra de Localidade para visibilidade pública.

11.1 Regras principais

- Cliente não informa recrutadorId; o backend obtém o recrutador pelo JWT.
- Criação, atualização e exclusão devem validar ownership.
- TipoDeEmprego usado na vaga deve existir e estar aprovado.
- Publicar como empresa exige vínculo aprovado.
- Oportunidade pública exige Localidade validada, conforme seção 9.
- Oportunidade pendente pode aparecer no /me do recrutador, mas não no público.
- Em /api/empresas/{id}/oportunidades, o escopo vem do path e não de query param.

11.2 Visões da oportunidade

Visão	Descrição	Cuidados
Pública	Lista e detalhe de vagas disponíveis para navegação geral.	Não expõe pendentes nem detalhes técnicos.
Recrutador /me	Painel das vagas criadas pelo recrutador autenticado.	Pode mostrar status de localidade e pendências próprias.
Candidato /me	Vagas em que o candidato demonstrou interesse.	Escopo pertence ao candidato autenticado.
Empresa pública	Oportunidades associadas à empresa do path.	Empresa e vagas precisam estar publicamente válidas.

11.3 Interesse em oportunidade

O interesse conecta candidato e vaga. O candidato autenticado pode demonstrar ou remover interesse; duplicidade é bloqueada; remoção de interesse inexistente é regra de negócio. O frontend não deve enviar usuariId para esse fluxo.

11.4 Cuidados futuros

Novos fluxos como pausar, arquivar, republicar, duplicar ou alterar contexto de publicação devem preservar ownership e regras de vínculo com empresa. Alterar uma vaga pessoal para vaga de empresa não deve ser tratado como simples update de campo sem validação de permissão.

12. Candidato, experiências, interesses e indicações

Esses módulos representam a parte da API voltada à pessoa que busca oportunidades. A refatoração reorganizou os fluxos pessoais em torno do usuário autenticado, reduzindo IDOR e separando identidade de papel de negócio.

12.1 PerfilCandidato

PerfilCandidato representa o papel do usuário como candidato. Ele concentra experiências profissionais, interesses em vagas e base para futuras preferências de empregabilidade. A regra é que recursos pessoais do candidato devem usar /me e não usuariold no path.

12.2 Experiências profissionais

- Experiência pertence ao candidato autenticado.
- Candidato gerencia suas experiências em fluxos pessoais.
- Admin pode ter visão global separada e protegida.
- Experiência se relaciona com TipoDeEmprego, preservando organização do histórico profissional.

12.3 Interesses em vagas

- Candidato autenticado demonstra, remove e lista interesses.
- Duplicidade é bloqueada.
- Remoção inexistente gera regra de negócio.
- Listagem de interesses é paginada e pertence ao candidato autenticado.
- statusLocalidade pode fazer sentido em contexto interno, sem alterar a regra pública.

12.4 Indicações

- Usuário autenticado pode indicar outro usuário.
- Autoindicação é proibida.
- Indicações dadas e recebidas continuam como List no MVP.
- Exclusão deve respeitar autoria.

12.5 Preferências

Preferências existem como base de domínio, mas não foram consolidadas como fluxo HTTP completo no MVP. Futuramente podem apoiar recomendações, match, alertas e filtros personalizados usando Tipos aprovados e Localidade validada.

13. Tipo de emprego e curadoria

TipoDeEmprego representa o catálogo de áreas, cargos ou categorias usadas em oportunidades, experiências, preferências e filtros. No estado final, ele passou a ser um catálogo moderado, separando tipos aprovados de sugestões não aprovadas.

Tipos aprovados formam o catálogo público. Tipos sugeridos por usuários entram como pendentes de aprovação.

13.1 Regras de curadoria

- Frontend público deve consumir apenas tipos aprovados.
- Usuário pode sugerir novo tipo, mas ele nasce não aprovado.
- Admin pode criar tipo aprovado, aprovar individualmente ou aprovar em lote.
- `/api/admin/tipos-emprego/nao-aprovados` impõe `aprovado=false` no backend.
- Oportunidade só deve usar TipoDeEmprego aprovado.
- Exclusão deve respeitar integridade quando o tipo estiver em uso.

13.2 Por que curadoria importa

Sem curadoria, o catálogo ficaria poluído por duplicidades, variações de escrita e categorias inadequadas. A moderação preserva qualidade de filtros, formulários, experiências e futuras recomendações.

13.3 Evoluções futuras

Possíveis melhorias incluem recusa com motivo, desativação em vez de exclusão, merge de duplicados, registro de quem aprovou, agrupamento por área e hierarquia de categorias.

14. Paginação e filtros

Paginação e filtros transformaram listagens simples em contratos adequados para uso real. O padrão adotado foi Page, Pageable, FilterDTO, Specification e JpaSpecificationExecutor nas listagens com risco de crescimento ou necessidade de busca.

14.1 Padrão adotado

Elemento	Papel
FilterDTO	Agrupa filtros opcionais do endpoint.
Pageable	Recebe page, size e sort da query string.
Page<ResponseDTO>	Retorna content e metadados de paginação.
Specification	Monta filtros combináveis e regras de visibilidade.
@ParameterObject	Ajuda o Swagger a documentar filtros e paginação.

14.2 Endpoints paginados principais

- GET /api/oportunidades
- GET /api/empresas
- GET /api/empresas/{id}/oportunidades
- GET /api/recrutadores/me/oportunidades
- GET /api/candidatos/me/interesses/vagas
- GET /api/admin/usuarios
- GET /api/admin/experiencias
- GET /api/admin/tipos-emprego/nao-aprovados

14.3 Regras que importam

- Não paginar em memória quando o banco pode paginar.
- Filtrar pendências no banco antes de paginar endpoints públicos.
- Query params filtram dentro do escopo; não devem alterar path ou token.
- Público filtra apenas recursos publicáveis; /me pode filtrar recursos internos.
- Filtros novos exigem atualização de FilterDTO, Specification, Swagger e testes.

14.4 Contrato futuro

No MVP, Page direto do Spring foi aceito. Futuramente, pode ser criado DTO próprio de paginação para estabilizar o JSON, mas isso deve ser alinhado com o frontend.

15. Tratamento de erros

Erro também é contrato da API. A refatoração substituiu respostas heterogêneas e exceptions genéricas por um tratamento global mais previsível para frontend e backend.

15.1 Estrutura final

- `StandardError` para erros gerais.
- `ValidationError` e `FieldMessage` para erros de validação por campo.
- Exceptions específicas para recurso inexistente, regra de negócio, conflito, duplicidade e integridade.
- `@RestControllerAdvice` para centralizar a conversão de exception em resposta HTTP.

15.2 Status principais

Status	Uso esperado
400	Payload inválido, parâmetro inválido ou erro de validação/bind.
401	Token ausente, inválido, expirado ou login inválido.
403	Usuário autenticado sem permissão.
404	Recurso inexistente ou indisponível no contexto público.
409	Conflito, duplicidade ou recurso em uso.
422	Regra de negócio violada com payload sintaticamente válido.

15.3 Regras especiais

- `LocalidadePendente` não é erro fatal de cadastro; pode resultar em sucesso com status interno.
- IDs estruturados de localidade inválidos ou inconsistentes podem gerar erro.
- Mensagens humanas devem estar em português correto e UTF-8 real.
- Frontend deve usar status HTTP para comportamento e message para feedback ao usuário.

16. Documentação, README, testes, Docker e Postman

A fase final preparou a API para ser entendida, executada, testada e integrada. A documentação foi distribuída por papéis, evitando que um único artefato tente resolver tudo.

16.1 Artefatos finais

Artefato	Função
Swagger/OpenAPI	Contrato HTTP: endpoints, parâmetros, DTOs, respostas, filtros e autenticação.
README	Visão geral, stack, execução local, Docker, Swagger e comandos principais.
JavaDoc técnico	Decisões internas próximas ao código sensível.
docs/testing.md	Organização e execução da suíte de testes.
Postman	Testes manuais, login e fluxos principais.
Docker	Reprodutibilidade local e apoio à demonstração.

16.2 Cuidados de manutenção

- Contrato mudou, Swagger muda.
- Forma de executar mudou, README muda.
- Teste ou suíte mudou, docs/testing.md muda.
- Fluxo principal mudou, Postman deve ser revisado.
- Decisão arquitetural sensível mudou, JavaDoc e documentação histórica devem acompanhar.

16.3 Limites

Docker melhora execução local, mas não equivale a produção completa. A suíte de testes cresceu, mas ainda pode evoluir com cobertura de corpos de erro, JWT inválido/expirado, CORS, profiles e writes em controllers.

17. Guia de integração para o frontend

O frontend deve escolher endpoints pelo contexto da tela, não apenas pelo nome da entidade. A API trabalha com três contextos: público, /me e admin.

17.1 Regras gerais

- Login em POST /login retorna JWT no header Authorization.
- Endpoints protegidos usam Authorization: Bearer <token>.
- 401 significa autenticação ausente/inválida; 403 significa usuário sem permissão.
- Telas pessoais usam /me e não enviam usuariold.
- Telas admin usam /api/admin/** e exigem perfil ADMIN.
- Endpoints paginados retornam dados em content.
- Filtros devem ser enviados por query params.
- DTO público e DTO interno podem ter campos diferentes.
- Swagger confirma o contrato exato; Postman valida fluxos manualmente.

17.2 LocalidadePendente no frontend

Recursos com LocalidadePendente podem existir internamente e não aparecer publicamente. Em telas internas, o frontend deve informar que o recurso foi salvo, mas aguarda validação de localidade para exposição pública. Não prometa confirmação manual ou notificação automática se esses fluxos ainda não existirem.

17.3 Localidade estruturada

Sempre que possível, o frontend deve usar IDs de país, estado e cidade vindos dos catálogos de Localidade. Texto livre aumenta a chance de pendência e deve ser tratado como fluxo sujeito à validação.

18. Contratos importantes para telas

Esta seção resume quais contratos e regras cada tipo de tela precisa observar. Os detalhes de URL, método, DTO e status devem ser conferidos no Swagger.

Tela	Contexto	Regras importantes
Login	Público	Capturar JWT no header Authorization.
Cadastro	Público	Cria conta; login é fluxo separado. Usuário não escolhe role admin.
Perfil do usuário	/me	Usar token como identidade; não enviar usuarioid.
Oportunidades públicas	Público	Retorna apenas vagas com Localidade validada; resposta paginada em content.
Detalhe público da oportunidade	Público	Pode retornar 404 se a vaga existir internamente, mas não estiver publicável.
Empresas públicas	Público	Retorna apenas empresas com Localidade validada.
Oportunidades da empresa	Público	Empresa vem do path; query param não altera escopo.
Painel do recrutador	/me	Lista próprias vagas, inclusive pendentes; pode exibir statusLocalidade.
Criação de oportunidade	/me recrutador	Tipo aprovado, modalidade, localidade e vínculo aprovado se publicar como empresa.
Painel do candidato	/me candidato	Experiências, interesses e indicações pertencem ao usuário autenticado.
Tipos de emprego	Público/Admin	Público consome aprovados; admin modera pendentes.
Admin	/api/admin/**	Exige ADMIN; listagens costumam ser paginadas e filtráveis.
Localidade	Catálogo público	Usar IDs estruturados para filtros e formulários sempre que possível.

18.1 Decisão rápida de endpoint

Tela aberta usa endpoint público. Tela do usuário logado usa /me. Tela de gestão global usa /api/admin/**.

19. Decisões de MVP e trade-offs

O MVP priorizou segurança, coerência de domínio, contratos claros e visibilidade pública confiável. Algumas funcionalidades foram conscientemente adiadas para evitar complexidade prematura.

Decisão	Trade-off
Salvar recurso com LocalidadePendente e ocultar do público	Menos bloqueio no cadastro, mas recurso pode não aparecer imediatamente.
Não implementar confirmação manual de localidade no MVP	Evita fluxo complexo de UX, mas deixa pendências dependentes de reprocessamento.
Não notificar automaticamente sobre pendência	Evita infraestrutura de e-mail/notificação, mas exige feedback por tela interna.
Empresa sem ownership formal	Permite vínculo moderado no MVP, mas governança empresarial fica para depois.
Page direto do Spring	Entrega rápida, mas contrato JSON pode ser estabilizado futuramente.
Nem tudo paginado	Simplifica catálogos/listas pequenas, mas pode exigir migração futura.
DTO interno expõe status técnico de localidade	Ajuda o dono agora, mas pode ser refinado para UX no futuro.
TipoDeEmprego moderado	Mais controle do catálogo, mas exige fluxo admin.
JWT stateless sem blacklist	Simples para MVP, mas logout/revogação efetiva ficam para depois.

Trade-off documentado não é dívida invisível. É decisão consciente de escopo.

20. Roadmap pós-MVP

O pós-MVP deve evoluir a API sem desfazer as decisões centrais: /me para fluxos pessoais, Localidade validada para público, LocalidadePendente como fila técnica, curadoria de TipoDeEmprego e separação entre público, /me e admin.

Evolução	Motivo	Cuidado principal
Confirmação manual de localidade	Resolver ambiguidades com participação do usuário.	Não escolher localidade automaticamente.
Notificação de recurso oculto	Informar usuário quando empresa/vaga não aparecer publicamente.	Não tratar pendência como erro fatal.
Ownership formal de Empresa	Definir dono, administradores e permissões empresariais.	Não confundir vínculo aprovado com ownership.
Painel operacional de LocalidadePendente	Acompanhar, reprocessar e resolver pendências.	Manter pendência como fila técnica, não recurso público.
Contrato próprio de paginação	Estabilizar JSON de Page.	Alinhar migração com frontend.
Revisão dos DTOs internos de localidade	Reduzir exposição técnica e melhorar UX.	Preservar dados necessários ao dono.
Revogação de token	Logout efetivo e invalidação após eventos críticos.	Planejar storage e impacto no filtro JWT.
Preferências do candidato	Base para match, alertas e recomendações.	Usar /me, tipos aprovados e Localidade validada.
Infraestrutura de produção	Banco persistente, secrets, CI/CD, logs, healthchecks e backups.	Docker local não substitui produção.
Migrações de banco	Versionar evolução de schema em ambiente persistente.	Avaliar Flyway ou Liquibase.

21. Guia para o próximo desenvolvedor backend

Antes de alterar a API, entenda qual problema a regra atual resolveu e qual impacto a mudança terá no frontend, nos testes e na documentação.

21.1 Regras que não podem ser quebradas

- Não expor entidade JPA como contrato HTTP.
- Não colocar regra de negócio relevante no controller.
- Não substituir /me por userId em fluxos pessoais.
- Não confiar apenas no frontend para segurança.
- Não expor LocalidadePendente em endpoints públicos.
- Não chamar Google Maps em controller ou listagem pública.
- Não escolher localidade ambígua automaticamente.
- Não tratar vínculo aprovado como ownership de Empresa.
- Não publicar como empresa sem vínculo aprovado.
- Não permitir oportunidade sem recrutador autenticado como dono.
- Não misturar tipos aprovados e não aprovados no catálogo público.
- Não paginar em memória quando o banco pode paginar.
- Não usar query param para controlar escopo que vem do token ou path.
- Não lançar RuntimeException genérica para regra de negócio esperada.

21.2 Checklist antes de commit relevante

- Controller continua fino?
- Service protege regra de negócio e ownership?
- DTO foi criado/ajustado sem expor entidade?
- Público, /me e admin estão separados?
- LocalidadePendente continua oculta do público?
- Filtros estão na Specification e não em memória?
- Erros usam exceptions e handler global?
- Swagger, README, docs/testing.md ou Postman precisam de atualização?
- Testes foram criados ou ajustados?
- Textos humanos estão em UTF-8 com acentuação correta?

21.3 Ordem recomendada de estudo

9. README e Swagger.
10. Segurança e endpoints /me.
11. Localidade e LocalidadePendente.
12. Empresa, Recrutador e Oportunidade.
13. Candidato, experiências, interesses e indicações.
14. TipoDeEmprego e curadoria.
15. Paginação, erros, testes e documentação.

22. Guia para o desenvolvedor frontend

Este guia final funciona como checklist de integração. As explicações completas estão nas seções anteriores; aqui ficam os cuidados práticos antes de implementar telas.

22.1 Checklist por tema

Tema	Cuidados
Autenticação	Capturar JWT do header Authorization no login e enviar como Bearer Token.
401/403	401 leva ao login; 403 mostra falta de permissão.
/me	Não enviar usuarioid em fluxos pessoais. Token define o usuário.
Admin	Usar /api/admin/** apenas em telas de gestão e tratar 403.
Page	Ler lista em content; page começa em zero; filtros vão por query params.
Localidade	Preferir IDs de país/estado/cidade. Texto livre pode gerar pendência.
LocalidadePendente	Não é erro fatal. Recurso pode aparecer internamente e não no público.
DTOs	Não esperar mesmos campos em endpoint público e interno.
TipoDeEmprego	Formulários comuns usam catálogo de aprovados. Sugestões dependem de admin.
Empresa	Solicitar vínculo não significa vínculo aprovado; recrutador vinculado não é dono.
Oportunidade	Publicar como empresa exige vínculo aprovado; vaga pública exige Localidade validada.
Erros	Usar status HTTP para comportamento e mensagem do backend para feedback.
Swagger/Postman	Confirmar contrato no Swagger e validar fluxo no Postman antes de telas complexas.

22.2 Perguntas antes de implementar uma tela

- A tela é pública, /me ou admin?
- O endpoint exige token?
- O retorno é Page ou List?
- Existe filtro no backend para essa busca?
- O recurso pode estar pendente por Localidade?
- A tela está prometendo alguma funcionalidade que ficou para pós-MVP?
- O Swagger confirma o contrato exato?
- O fluxo funciona no Postman?

Boa UX não ignora regra de backend; ela explica a regra para o usuário.